FROM TOKENS TO OPERATORS & OPERANDS

- PLEASE NOTE, WHEN I FIRST WROTE THESE DOCUMENTS I ACCIDENTALLY MIXED-UP 'OPERATORS' AND 'OPERANDS' FOR THE FIRST FEW CHAPTERS.

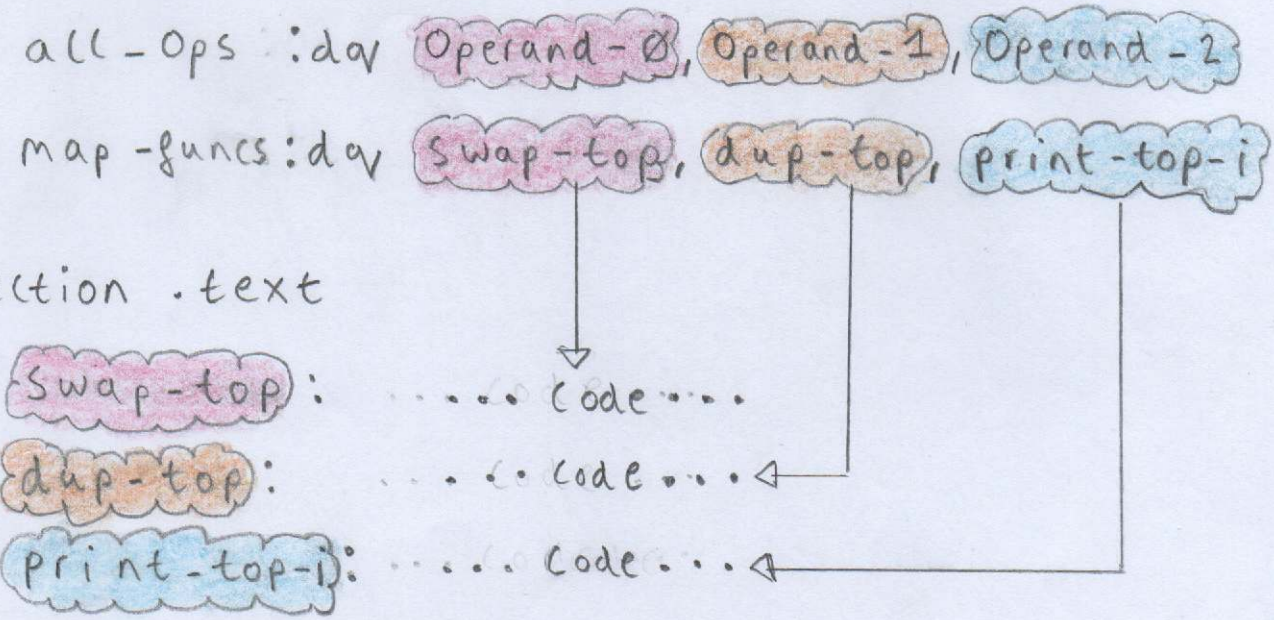
- I FIXED THIS IN LATER CHAPTERS.

I have the whole input file stored like an **ARGV. All of the comments have been removed so Now I can loop through this and push tokens onto the stack and when I get an operand, I can run the function for it. This was simple to set up in the .data segment.

```
section .data
    Operand-0: db "swap", 0
    Operand-1: db "dup", 0
    Operand-2: db "print-i", 0

    all-ops :dq Operand-0, Operand-1, Operand-2
    map-funcs:dq swap-top, dup-top, print-top-i

section .text
    swap-top:    ....code....
    dup-top:     ....code....
    print-top-i: ....code....
```



all-Ops: is an array of pointers to strings. I can use this array and iterate through it to find any operand we come across from our [argvPtr]. By iterating through the array, I can find an index that matches the string I am looking for.

```
[all-Ops + r8 * 0x08]   ; r8 is my index
```

I use my own string compare function and if there is a match, I use this index to call the matching function. If r8 has a value of 0x01 when we iterated through the loop and it matched "dup" string, I can use r8 to call the matching function, call dup-top.

I wrote a function check-is-operand and this returns true / false if the string matches an operand, r8 is an index here in the code.

```
Mov     rdx, [argv Ptr]

.Looping:

Mov     rdi, [ rdx + r8 * 0x08]
Call    check-is-operand ; rax = 1 if operand, 0 = token

test    rax, rax
jz      .pushOn

Call    [map-funcs + r8 *0x08]; God operand, call it's fuction
jmp     .next Argv

.pushOn:
Mov     rdi, [ rdx + r8 * 0x08]
call    push-token ; this pushes the token onto the stack

.next Argv:
inc     r8
jmp     .Looping
```

Wherever I find a string that matches an operand, I can easily call the function to carry out work on the stack.

The operand is NEVER pushed onto the stack.